Demonstrations

Lerinckx Maxime

May 2025

1 Algorithme ElGamal

$y^2 \equiv x^3 + ax + b \pmod{\text{mod}}$
G Point générateur
$S_{k,A} = \text{random}(\llbracket 1, \text{mod} \rrbracket)$
$S_{k,B} = \text{random}(\llbracket 1, \text{mod} \rrbracket)$

$P_{k,A} = S_{k,A} \cdot G$
$P_{k,B} = S_{k,B} \cdot G$
$S_{k,B}P_{k,A} = S_{k,B} \cdot P_{k,A}$
$M_{chiffre} = S_{k,B}P_{k,A} + M$
M message transformé en un point

Alice reçoit $(M_{chiffre}, P_{k,B})$

$S_{k,A}P_{k,B} = S_{k,A} \cdot P_{k,B}$
$M_{dechiffre} = M_{chiffre} - S_{k,A}P_{k,B}$

On retrouve que $M = M_{dechiffre}$ car :
$M_{dechiffre} = M_{chiffre} - S_{k,A}P_{k,B}$
$= S_{k,B}P_{k,A} + M - S_{k,A}P_{k,B}$
$= S_{k,B} \cdot P_{k,A} + M - S_{k,A} \cdot P_{k,B}$
$= M + S_{k,B} \cdot (S_{k,A} \cdot G) - S_{k,A} \cdot (S_{k,B} \cdot G)$
$= M + S_{k,A} \cdot S_{k,B} \cdot G - S_{k,A} \cdot S_{k,B} \cdot G$
$= M$

2 Échange de clés Diffie-Hellman

$S_{k,A} = \text{random}([\![1, \text{mod}]\!])$
$S_{k,B} = \text{random}([\![1, \text{mod}]\!])$

$P_{k,A} = S_{k,A} \cdot G$
$P_{k,B} = S_{k,B} \cdot G$

Alice : $K = S_{k,A} \cdot P_{k,B}$
Bob : $K' = S_{k,B} \cdot P_{k,A}$

$K = S_{k,A} \cdot (S_{k,B} \cdot G) = S_{k,B} \cdot (S_{k,A} \cdot G) = K'$

3 Logarithme discret sur courbe elliptique

Soit p un nombre premier, et $\mathbb{F}_p$ le corps fini à p éléments.
Soient $a, b \in \mathbb{F}_p$ tels que $4a^3 + 27b^2 \not\equiv 0 \pmod{p}$.
$E(\mathbb{F}_p) = \left\{(x, y) \in \mathbb{F}_p^2 \mid y^2 \equiv x^3 + ax + b \pmod{p}\right\} \cup \{\mathcal{O}\}$
Soit $G \in E(\mathbb{F}_p)$ un point générateur d'un monogène fini de $E(\mathbb{F}_p)$ d'ordre n
$\langle G \rangle = \{k \cdot G \mid k \in [\![0, n-1]\!]\}$
Donc $\forall P \in \langle G \rangle, \exists! \, k \in [\![0, n-1]\!]$ tel que $P = k \cdot G$
k est appelé logarithme de P en base G et est noté $\log_G(P) = k$

Le problème du logarithme discret sur les courbes elliptiques est :
Trouver $k \in [\![0, n-1]\!]$ tel que $P = k \cdot G$ connaissant G et P
i.e on cherche $log_G(P)$

Il existe deux grand algorithmes conçues pour résoudre le problème du logarithme discret :

3.1 Baby-step Giant-step (Daniel Shanks)

L'algorithme résout le DLP $g^x = h$ en $\mathcal{O}(\sqrt{N} \cdot \log N)$ étapes

Dans notre cas : G d'ordre $N, P \in \langle G \rangle, \exists! k < N$ tel que $P = k \cdot G$

① $n = \lfloor \sqrt{N} \rfloor + 1$
② Baby-step : $[i \cdot G \mid i \in [\![0, n]\!]]$
Giant-step : $[P + (j \cdot n) \cdot (-G) \mid j \in [\![0, n]\!]]$
③ Trouver (i, j) tel que $i \cdot G = P + (n \cdot j) \cdot (-G)$
④ $k = i + n \cdot j$ solution du DLP

Correction :
$P \in \langle G \rangle \Rightarrow \exists k \in [\![0, N-1]\!]$ tel que $P = k \cdot G$
Par Division Euclidiene de k par n : $\exists q \in [\![0, N-1]\!]$ et $r \in [\![0, n-1]\!]$ tel que $K = n \times q + r$ donc :

① comme $K < N$ et $N > \sqrt{N}$
on a : $q = \dfrac{k-r}{n} < \dfrac{N}{n} < n$ car $n - 1 \leq \sqrt{N} < n \implies N < n^2$
d'où $r \cdot G = P + (n \times q) \cdot (-G)$ avec $0 \leq r < n$ et $0 \leq q < n$

Complexité :
Le calcule des deux listes dans ② requiert $3n$ opération de groupe

On trie les deux listes pour trouver un élément commun, ce qui se fait en $\mathcal{O}(n \cdot \log n)$ étapes, or $n \approx \sqrt{N}$ d'où la compléxité totale est en $\mathcal{O}(\sqrt{N} \cdot \log N)$

3.1.1 Remarque

Même si l'algorithme Baby-step Giant-step est théoriquement plus rapide que les attaques naïves (*complexité* $\mathcal{O}(\sqrt{N} \log N)$), il reste impraticable pour les groupes utilisés en cryptographie moderne :

- Pour un ordre $N \approx 2^{256}$ (sécurité standard), $\sqrt{N} = 2^{128}$ opérations sont nécessaires.

- Cela représente environ 10^{38} opérations **et** autant d'espace mémoire (environ 10^{40} octets) pour stocker les résultats.

3.2 Algorithme de Pohlig-Hellman Avec les mêmes notations

① On décompose N en facteur premier : $N = \prod_{i=1}^{r} p_i^{e_i}$

② Pour chaque $p_i^{e_i}$:

Calcule k_i tel que $P_i = k_i \cdot G_i$ avec : $P_i = \frac{N}{p_i^{e_i}} \cdot P$ et $G_i = \frac{N}{p_i^{e_i}} \cdot G$

Utiliser une méthode approprié comme Baby-step Giant-step pour determiner k_i

③ Combinaison des resultats : utiliser le théorème des restes chinois pour reconstuire k à partir des k_i :
On cherche une solution du système $S = \{K \equiv k_i \pmod{p_i^{e_i}}\}_{i \in [\![1,r]\!]}$